

Unity Multiplayer Developer Technical Assessment

Unity NGO + Edgegap Dedicated Servers | Mobile Client Target

Objective

Build a small real-time multiplayer prototype in Unity for a mobile client target using:

- Unity Netcode for GameObjects (NGO)
- Unity Transport or an appropriate NGO-compatible transport
- A dedicated authoritative Unity server
- Edgegap for dedicated-server hosting
- A mobile client build targeting Android or iOS

The purpose of this assessment is to evaluate your ability to build a server-authoritative Unity multiplayer experience for mobile clients and deploy, configure, and validate its dedicated server on Edgegap.

This is not an advanced DevOps or cloud-infrastructure assessment. However, practical Edgegap integration is a required part of the end-to-end multiplayer solution and will be evaluated.

We are primarily evaluating:

- Server-authoritative gameplay
- NGO architecture
- NetworkVariables and RPC usage
- NetworkObject ownership
- Networked state machines
- Race-condition handling
- Multiplayer movement
- Physics and hit validation
- Disconnect handling
- Dedicated-server deployment and Edgegap integration

- Edgegap runtime configuration and deployment-specific mobile client connectivity
- Mobile touch input and interaction flow
- Mobile latency, bandwidth, and runtime-efficiency considerations
- Code quality

Visual quality is not important. Primitive Unity objects and simple mobile UI controls are sufficient; the focus is networking, gameplay correctness, Edgegap integration, and a functional mobile experience.

Mobile Target Requirement

The intended game client platform is mobile. Submit a working client build for at least one mobile platform: Android or iOS.

At least one client in the recorded Edgegap demonstration must be the mobile build running on a physical device. The remaining clients may run as additional mobile builds, standalone desktop test builds, or Unity Editor instances.

The mobile client must use touch-oriented gameplay controls. Keyboard/mouse controls may be retained for Editor testing, but they must not be the only way to play the submitted mobile build.

The mobile client must connect to the Edgegap-hosted dedicated server using the deployment's public connection information; the solution must not depend on localhost, a LAN-only address, or an Editor-only connection path.

1. Multiplayer Setup

Create a prototype supporting:

- 8 players maximum
- Two teams
- Dedicated server
- Multiple connected Unity clients, including a mobile client build
- Touch-based player input suitable for the submitted Android or iOS build
- Networked player movement
- Several collectible objective objects
- One shared throwable object

- Player elimination
- Server-authoritative match completion

The server must run independently from the clients.

Clients must never act as host.

2. Unity NGO Requirements

Use Unity Netcode for GameObjects.

Demonstrate correct usage of appropriate NGO concepts including:

- NetworkManager
- NetworkObject
- NetworkBehaviour
- NetworkVariable
- RPCs
- Ownership
- Spawn/despawn
- Connection callbacks
- NGO-compatible transport

You should be able to explain why each networking mechanism was selected.

3. Dedicated Server Authority

Important gameplay logic must execute or be validated by the dedicated server.

Clients may request actions.

The server determines the result.

The server should be authoritative over:

- Movement validity
- Pickups
- Object ownership

- Interaction state
- Timed interaction completion
- Shared-object possession
- Throws
- Hits
- Elimination
- Match victory

Clients should not directly determine important gameplay outcomes.

4. Player Movement

Implement multiplayer movement suitable for a mobile client.

Requirements:

- Local touch input on mobile (for example, virtual joystick/buttons or another clear touch scheme)
- Server-authoritative gameplay position
- Smooth remote movement
- Reasonable responsiveness and remote smoothing under normal internet/mobile-network latency
- Avoid unnecessary per-frame RPC traffic or other obviously wasteful network messaging
- Keep the mobile client reasonably efficient in network traffic, allocations, and update work for the scope of this prototype

Be prepared to explain:

- Input flow
- Transform synchronization
- Interpolation
- Prediction, if used
- Reconciliation, if used
- Movement validation

The goal is not to build a complex competitive-shooter movement stack or production-level mobile optimization.

We want to see correct multiplayer architecture with sensible mobile input, latency handling, and network efficiency.

5. Objective Object State Machine

Create several networked collectible objects.

Each should have a state such as:

Available

Carried

InteractionInProgress

Completed

The server controls state transitions.

Example:

Available

→ Pickup request

→ Server validation

→ Carried

Then:

Carried

→ Interaction started

→ Server timer

→ Completed

Invalid interactions should return the object to an appropriate state.

6. Race-Condition Test

Two clients should attempt to obtain the same objective nearly simultaneously.

Only one may succeed.

The server must resolve the conflict.

All clients must eventually see the same authoritative result.

Be prepared to explain how the implementation prevents:

- Duplicate possession
 - Duplicate completion
 - Conflicting ownership
 - Client-side state divergence
-

7. Timed Interaction

Implement a server-authoritative timed interaction lasting approximately one second.

The server should cancel the interaction if the player:

- Moves away
- Loses the required object
- Is eliminated
- Becomes otherwise invalid

The client may visually predict/display progress using a simple mobile-friendly progress indicator.

Only the server determines successful completion.

8. Shared Throwable Object

Implement one shared object that may only be possessed by one eligible player.

Implement:

- Pickup
- Possession
- Transfer
- Throw
- Recovery

The server controls possession.

When the player possesses this object, apply a movement restriction.

The server must enforce the restriction.

9. Throw and Hit Validation

The client sends a request to throw from a mobile-friendly input action; the server remains authoritative.

The dedicated server validates:

- Ownership
- Player state
- Throw state
- Direction
- Timing/cooldown

The server determines the actual outcome.

The client cannot send:

“I hit Player X.”

The server determines collisions/hits.

10. Elimination

A valid server-confirmed hit eliminates the target for the current match.

After elimination:

- State is replicated to all clients
 - Player cannot interact
 - Player cannot collect objects
 - Server rejects gameplay requests
 - Player remains eliminated until the match ends
-

11. Match State Machine

Implement:

WaitingForPlayers

Starting

Playing

Finished

The dedicated server controls these states.

Two generic victory conditions should be implemented.

Condition A

A required number of objective objects have been successfully completed.

Condition B

All active players on the opposing team have been eliminated.

The server determines which condition occurred first.

12. Disconnect Handling

Correctly handle disconnects, including a mobile client losing connection or leaving the app, while the player is:

- Carrying an objective
- Performing a timed interaction
- Possessing the shared object
- Already eliminated
- Disconnected because the mobile app is closed, terminated, or loses network connectivity

No gameplay object should remain permanently owned by a disconnected player.

The match should remain valid. Rejoining the same session is not required, but server-side state and object ownership must remain correct after the mobile client disconnects.

13. Edgegap Dedicated Server Integration

Deploy the Unity dedicated server to Edgegap and demonstrate a complete mobile-client-to-Edgegap-to-server multiplayer flow.

Edgegap integration is a core assessment area. We are not evaluating advanced cloud architecture, but you should be comfortable with the practical deployment lifecycle needed to run and validate your dedicated server.

We want to verify that you can take a working Unity NGO dedicated server, deploy it through Edgegap, configure its networking correctly, obtain deployment-specific connection details, and connect a real mobile client plus additional test clients to it.

Requirements:

- Create a Unity Linux dedicated-server build suitable for headless hosting
- Configure the build to automatically start in dedicated-server mode
- Package and deploy the server through Edgegap; Edgegap tooling may be used
- Configure the game networking port and protocol correctly for the Edgegap deployment
- Verify that the Edgegap deployment starts successfully and reaches a usable running state
- Obtain the deployment's current public connection address and external port
- Connect at least 3 Unity clients to the deployed server and demonstrate gameplay; at least one must be the submitted mobile build running on a physical device
- Use deployment-specific connection details rather than a hardcoded public endpoint
- Verify the mobile build can reach the public Edgegap endpoint without relying on localhost, LAN-only addressing, or Editor-specific configuration
- Show the relevant Edgegap deployment status and/or runtime log output used to confirm server startup or diagnose a basic connection problem
- Be able to explain what must be updated on the client when a new Edgegap deployment provides different connection information

You may use Edgegap's Unity tooling or other supported Edgegap workflow where appropriate. A custom production orchestration layer is not required.

Candidate Hosting / Cost Note

Candidates are not expected to purchase hosting for this assessment. A paid Edgegap subscription is not required; use Edgegap's free trial or other no-cost testing allowance where available.

The server only needs to remain deployed long enough to complete testing and record the required multiplayer demonstration. Continuous or production hosting is not expected.

If Edgegap's free trial or no-cost testing option is unavailable or its terms have changed, contact the assessment reviewer before purchasing any hosting.

AWS GameLift or another hosting provider is not required for this assessment. The required deployment demonstration should use Edgegap as specified above.

14. Edgegap Runtime and Client Connection Configuration

The dedicated server and mobile client connection flow must correctly use runtime information from the Edgegap deployment.

The solution must not depend on a hardcoded public IP address or external port.

The mobile client must be able to receive or enter the deployment's current public address and external port through a simple test-friendly mechanism.

Do not assume the external port exposed by the deployment is identical to the server's internal listening port; configure and document the networking setup correctly.

A simple in-app mobile test interface, configuration file, deep-link/test configuration, or other clear mechanism for supplying deployment connection details is acceptable.

Document any startup arguments, environment configuration, or runtime settings required by the dedicated server.

Production matchmaking, authentication, autoscaling, and custom Edgegap API orchestration are not required unless you choose to implement them.

15. Networking, Mobile Client, and Hosting Decisions

Provide a short explanation covering the multiplayer architecture, mobile client considerations, and the Edgegap hosting flow:

Movement

How NGO synchronizes movement and how authority is handled.

Objective state

How persistent authoritative states are synchronized.

Requests

How clients request pickup, throw and interaction actions.

Events

How one-time effects and gameplay events are communicated.

Ownership

How NetworkObject ownership differs from gameplay possession in your architecture, if applicable.

Mobile input and performance

How touch input becomes network requests/input, and what you did to avoid unnecessary network traffic or mobile runtime overhead.

Mobile connectivity

How the mobile build receives the current Edgegap endpoint and what happens to authoritative gameplay state when that mobile client disconnects.

Edgegap deployment

How the dedicated-server build is started on Edgegap, which port/protocol it listens on, and how deployment-specific connection details reach the client.

Deployment validation

How you verify server startup and how you would diagnose a deployment that appears to be running but is not reachable by clients.

Required Demonstration

Demonstrate:

- Start or show an active Edgegap deployment running the submitted dedicated-server build
- Show the Edgegap deployment connection information used by the clients (public address and external port)
- Show evidence that the dedicated server started successfully, using deployment status and/or relevant runtime log output
- Connect at least 3 Unity clients using the current Edgegap deployment information; at least one must be an Android or iOS build running on a physical mobile device
- Demonstrate touch-based movement and at least one gameplay interaction from the mobile client
- Demonstrate that client connection settings are not tied to a hardcoded public endpoint
- Server-authoritative movement
- Objective pickup
- Simultaneous pickup attempt
- Timed interaction
- Shared object possession
- Throw
- Server-authoritative hit
- Player elimination
- Disconnect a client while possessing an object; using the mobile client for this scenario is encouraged

- Match completion
-

Deliverables

Submit:

1. Unity project/source
2. Mobile client build for at least one platform (Android or iOS)
3. Unity dedicated-server build
4. README
5. Short networking architecture explanation
6. Short demonstration video showing the Edgegap deployment/connectivity, the mobile build running on a physical device, and multiplayer gameplay

The README should explain:

- How the dedicated-server build is configured and started
 - How the server is packaged/deployed to Edgegap and how to start or recreate a deployment
 - Which game port/protocol is used and how the Edgegap public address/external port are obtained
 - How the mobile client and other test clients are configured to connect using the current Edgegap deployment information
 - How to reproduce the multiplayer test with at least 3 clients
 - Which mobile platform/device and OS version were used for testing, and how to install/run the submitted mobile build
 - How to verify server startup and where to check basic deployment status/log output
 - Any known Edgegap deployment or connectivity limitations and basic troubleshooting notes
 - Do not include Edgegap API tokens, credentials, or other secrets in the submitted project, README, or video
-

Evaluation

Server Authority and Validation — 20%

Unity NGO Architecture — 15%

Object Ownership and State Synchronization — 15%

Mobile Input, Movement, Latency, and Client Efficiency — 10%

Throwing and Hit Validation — 10%

Disconnect, Mobile Lifecycle, and Network Edge Cases — 5%

Edgegap Deployment, Runtime Configuration, and Mobile Client Connectivity — 20%

The Edgegap portion evaluates:

- Successful deployment and startup of the submitted dedicated-server build
- Correct game port/protocol and runtime configuration
- Correct handling of deployment-specific public address and external port information
- Successful mobile/client connectivity through Edgegap without hardcoded public endpoints or LAN-only assumptions
- Ability to verify startup and diagnose basic deployment/connectivity issues
- Clear deployment and reproduction documentation
- Evidence that the submitted mobile build connects to and plays correctly through the Edgegap deployment

Code Quality and Documentation — 5%

Edgegap integration and mobile-client readiness are core parts of this assessment, alongside Unity multiplayer engineering.

The assessment evaluates your ability to build authoritative multiplayer gameplay with Unity NGO for a mobile client, carry the dedicated server through an end-to-end Edgegap deployment, and validate real mobile connectivity, runtime configuration, gameplay, and disconnect behavior. Advanced cloud architecture, production DevOps automation, production-grade mobile UI, and deep device optimization remain out of scope.